

Reconhecimento de Emoções Faciais em Tempo Real com NAS via Optuna e YOLOv8: Uma Abordagem com Validação Cruzada e Restrições de GPU em Nuvem

Wesley¹, Yure¹, Osevaldo¹, Guilherme¹, Arnaud¹

¹Programa de Pós-Graduação em Ciência da Computação
Universidade Federal do Maranhão (UFMA)
São Luís – MA – Brasil

{wesley, yure, osevaldo, guilherme, arnaud}@discente.ufma.br

Abstract. *This work describes the development of a real-time facial emotion recognition system, built as the final project for the Computer Vision Systems course at the Graduate Program in Computer Science of UFMA. The system combines a convolutional neural network trained from scratch on the FERPlus dataset with automatic architecture search via Neural Architecture Search using the Optuna framework with 3-fold cross-validation, and real-time face detection with a fine-tuned YOLOv8 model. The paper documents the technical decisions, the challenges faced with GPU restrictions on the Kaggle platform, the regularization strategies adopted to reduce overfitting, and the results obtained, including a validation accuracy of 66.78% on the FERPlus dataset with 8 emotion classes. The final integration was performed via OpenCV, enabling real-time emotion classification on webcam frames with simultaneous multi-face detection.*

Resumo. *Este trabalho descreve o desenvolvimento de um sistema de reconhecimento de emoções faciais em tempo real, desenvolvido como projeto final da disciplina de Sistemas de Visão Computacional do Programa de Pós-Graduação em Ciência da Computação da UFMA. O sistema combina uma rede neural convolucional treinada do zero sobre o dataset FERPlus com busca automática de arquitetura via Neural Architecture Search utilizando o framework Optuna com validação cruzada de 3 folds, e detecção de rostos em tempo real com o modelo YOLOv8 submetido a fine-tuning. O artigo documenta as decisões técnicas, os desafios enfrentados com restrições de GPU na plataforma Kaggle, as estratégias de regularização adotadas para redução de overfitting e os resultados obtidos, incluindo acurácia de validação de 66,78% no dataset FERPlus com 8 classes de emoções. A integração final foi realizada via OpenCV, permitindo classificação de emoções em frames de webcam em tempo real com detecção de múltiplos rostos simultâneos.*

1. Introdução

O reconhecimento automático de emoções faciais (FER, do inglês *Facial Expression Recognition*) é uma área de crescente relevância em visão computacional, com aplicações

em interfaces humano-computador, saúde mental, educação e sistemas de vigilância [Zhao et al. 2024]. A tarefa consiste em identificar, a partir de imagens ou vídeos de rostos humanos, estados afetivos como raiva, nojo, medo, felicidade, tristeza, surpresa, desdém e neutralidade.

O presente trabalho foi desenvolvido como projeto final da disciplina de Sistemas de Visão Computacional, com o objetivo de construir um sistema completo de FER em tempo real, desde a busca automática de arquitetura até a integração com câmera. O sistema utiliza o dataset FERPlus [Barsoum et al. 2016], que é uma extensão anotada por múltiplos avaliadores do dataset FER2013, contendo 8 classes de emoções em imagens grayscale de 48x48 pixels.

A principal contribuição deste trabalho se apoia na combinação de três abordagens: (1) uso do Optuna [Akiba et al. 2019] para Neural Architecture Search com validação cruzada de 3 folds sobre o conjunto de treino do FERPlus; (2) aplicação de estratégias de regularização para controle de overfitting; e (3) integração com o YOLOv8 [Terven and Córdova-Esparza 2023] submetido a fine-tuning para detecção de rostos em tempo real via webcam. Além disso, o artigo documenta os desafios práticos enfrentados com restrições de GPU na plataforma Kaggle, tais como limite de horas semanais e perda de sessões, que condicionaram significativamente as decisões de projeto.

O restante do artigo está organizado da seguinte forma: a Seção 2 apresenta a revisão da literatura; a Seção 3 descreve a metodologia adotada; a Seção 4 apresenta os resultados obtidos; e a Seção 5 encerra o trabalho com as considerações finais e direções para trabalhos futuros.

2. Revisão da Literatura

2.1. Reconhecimento de Emoções Faciais com CNNs

O reconhecimento de emoções faciais com redes neurais convolucionais tem evoluído significativamente desde a publicação do dataset FER2013. [Barsoum et al. 2016] introduziram o FERPlus, estendendo as anotações originais com distribuições de rótulos obtidas por crowdsourcing com 10 avaliadores por imagem, reduzindo o ruído de anotação e adicionando a classe *contempt* às 7 emoções básicas. O dataset tornou-se um benchmark amplamente utilizado, com modelos recentes atingindo acurácias superiores a 90% com arquiteturas avançadas como transformers e redes de atenção dual [Zhao et al. 2025][Hua et al. 2025].

Apesar do progresso, a literatura reconhece que a generalização de modelos treinados em FERPlus para condições reais de uso é desafiadora. Fatores como variação de pose, iluminação e distância da câmera afetam significativamente o desempenho em ambientes não controlados [Fang et al. 2023]. Este trabalho se insere nesse contexto, treinando uma CNN do zero sem transfer learning, como parte dos critérios exigidos pela proposta metodológica do projeto final da disciplina.

2.2. Neural Architecture Search com Optuna

O NAS (*Neural Architecture Search*) é a área de AutoML voltada à automatização do processo de projeto de arquiteturas de redes neurais [Wang and Zhu 2024]. Em vez de definir manualmente o número de camadas, filtros e hiperparâmetros, o NAS explora

automaticamente o espaço de possíveis arquiteturas em busca da mais adequada para uma tarefa e dataset específicos [Wu and Tsai 2024].

O Optuna [Akiba et al. 2019] é um framework de otimização de hiperparâmetros baseado em amostragem bayesiana que implementa o algoritmo *Tree-structured Parzen Estimator* (TPE). Sua interface de sugestão dinâmica de hiperparâmetros o torna adequado para NAS, permitindo que a arquitetura da rede seja parametrizada como um espaço de busca e otimizada ao longo de múltiplas tentativas (*trials*). Estudos recentes demonstram que o NAS supera arquiteturas projetadas manualmente em diversas tarefas de visão computacional [White et al. 2023].

2.3. Detecção de Rostos com YOLOv8

O YOLOv8, lançado pela Ultralytics em 2023, representa o estado da arte em detecção de objetos em tempo real, com arquitetura *anchor-free* e cabeça desacoplada que melhora a precisão sobre versões anteriores [Terven and Córdova-Esparza 2023]. Sua eficiência computacional o torna adequado para uso com webcam em hardware convencional.

O fine-tuning do YOLOv8 em datasets especializados de rostos, como o WIDER-Face, tem demonstrado resultados superiores à detecção com pesos genéricos treinados no COCO e é amplamente utilizado para treino e avaliação de detectores de rostos, contendo imagens com grande variação de escala, pose e oclusão [Yang et al. 2016].

Com o intuito atenuar as limitações de detecção em condições típicas de uso com webcam, o WIDERFace foi combinado com o Face-Detection-Dataset como um segundo dataset de rostos centralizados e em close. [Yisihak and Li 2024].

3. Metodologia

3.1. Dataset e Pré-processamento

O dataset utilizado é o FERPlus [Barsoum et al. 2016], disponível no Kaggle na versão organizada por Arnab Kumar Roy, com imagens já separadas em três conjuntos: treino (66.379 imagens), validação (8.341 imagens) e teste (3.573 imagens), distribuídas em 8 pastas por classe. As imagens são grayscale de 48x48 pixels, cobrindo as seguintes emoções: raiva, desdenho, nojo, medo, felicidade, neutralidade, tristeza e surpresa.

As seguintes transformações foram aplicadas ao conjunto de treino via `torchvision.transforms`: conversão para escala de cinza, espelhamento horizontal aleatório com probabilidade 0,5, rotação aleatória de até 10 graus, translação aleatória de até 5% em cada eixo e normalização com média 0,5 e desvio padrão 0,5. Validação e teste receberam apenas redimensionamento e normalização, sem augmentation.

3.2. Busca de Arquitetura via NAS com Optuna

O NAS foi implementado com o framework Optuna em combinação com validação cruzada de 3 folds sobre o conjunto de treino. Para cada tentativa (*trial*), o Optuna sugere uma arquitetura parametrizada pelos seguintes hiperparâmetros:

- Número de camadas convolucionais: 1 a 4
- Número de filtros por camada: 16 a 128, passo 16
- Tamanho do kernel por camada: 3 ou 5

- Número de neurônios na camada densa: 64 a 256, passo 64
- Taxa de dropout: 0,2 a 0,5
- Taxa de aprendizado: 10^{-4} a 10^{-2} , escala logarítmica

Em cada tentativa, a arquitetura sugerida é avaliada nos 3 folds em rotação: na iteração 1, treina nos folds 2 e 3 e avalia no fold 1; na iteração 2, treina nos folds 1 e 3 e avalia no fold 2; na iteração 3, treina nos folds 1 e 2 e avalia no fold 3. Cada fold treina por 3 épocas, mantendo o tempo por tentativa em torno dos 13 minutos na GPU Tesla T4 do Kaggle no primeiro ciclo do experimento, com 30 tentativas, que durou 6 horas e 49 minutos. A métrica devolvida ao Optuna é a média das acurácias dos 3 folds.

O estudo foi executado em três ciclos principais de experimentos, condicionados pelas restrições de GPU da plataforma Kaggle, conforme detalhado na Subseção 3.5. O experimento do ciclo final atingiu 102 tentativas, com a melhor arquitetura proposta pelo Optuna alcançando acurácia média de 81,0% nos folds e uso mais de duas sessões de execução (i.e. 12 horas) da plataforma kaggle.

As arquiteturas propostas pelo Optuna ao longo das execuções convergiram para padrões estáveis: 4 camadas convolucionais, filtros em progressão crescente, camada densa enxuta e taxas de aprendizado na ordem de 10^{-3} . Os resultados detalhados de cada execução, incluindo os melhores trials e suas arquiteturas, são apresentados na Subseção 4.1.

3.3. Treino Final com Regularização

As arquiteturas derivadas do NAS foram treinadas do zero com o dataset completo de treino. Com autorização do orientador do projeto final, a equipe realizou ajustes manuais sobre as arquiteturas de saída do NAS, modificando filtros, kernels e parâmetros de regularização sem nova execução do Optuna, com o objetivo de verificar se a acurácia de validação melhorava. As seguintes estratégias de regularização foram adotadas progressivamente ao longo das iterações de melhoria:

- **Dropout2d** com taxa de 0,2 após cada camada convolucional, além do dropout padrão na camada densa.
- **AdamW** com weight decay de 10^{-4} no lugar do Adam original, para regularização L2 nativa.
- **Label smoothing** de 0,1 na função de perda `CrossEntropyLoss`, reduzindo a confiança excessiva do modelo nas previsões.
- **ReduceLROnPlateau** com fator 0,5 e paciência de 3 épocas, reduzindo a taxa de aprendizado quando a acurácia de validação estagnava.
- **Early stopping** com paciência de 12 épocas, encerrando o treino quando nenhuma melhoria de acurácia de validação era observada.
- **Pesos por classe na função de perda**: as classes raiva, nojo, medo e tristeza receberam peso 2,0, enquanto as demais receberam peso 1,0, motivado pela observação empírica de que essas emoções eram detectadas com menor frequência durante os testes com webcam.

O monitoramento do treino foi realizado via plataforma Weights and Biases (W&B), com envio automático de métricas a cada época e salvamento do modelo como artefato na nuvem, eliminando o risco de perda por encerramento de sessão na plataforma Kaggle.

3.4. Detecção de Rostos com YOLOv8

Para a detecção de rostos em tempo real, foi realizado o fine-tuning do YOLOv8 nano (`yolov8n.pt`) com um dataset combinado de dois conjuntos públicos no formato YOLOv5/v8: o WIDERFace [Yang et al. 2016] com 16.108 imagens de rostos em situações variadas, e o Face-Detection-Dataset com imagens de rostos centralizados e em close. A combinação resultou em 24.842 imagens de treino e 4.624 de validação.

A necessidade de combinar dois datasets surgiu da observação de que o fine-tuning realizado exclusivamente com o WIDERFace produzia baixa confiança de detecção em rostos frontais e próximos à câmera, padrão típico de uso com webcam. O Face-Detection-Dataset complementou com exemplos desse padrão específico, resultando em melhoria significativa da estabilidade do bounding box durante os testes.

O fine-tuning foi executado por 30 épocas com imagens de 640x640 e batch size de 16. O modelo resultante foi salvo como artefato no W&B para garantir rastreabilidade e reprodutibilidade.

3.5. Desafios com Restrições de GPU no Kaggle

A plataforma Kaggle oferece gratuitamente GPU Tesla T4 com cota de aproximadamente 30 horas semanais por conta e limite de 12 horas por sessão. Essas restrições condicionaram significativamente as decisões do projeto e geraram os desafios práticos descritos a seguir.

1. Perda de sessão durante o NAS. Nas primeiras execuções, o arquivo de banco de dados SQLite do Optuna era armazenado em `/kaggle/working`, diretório que é apagado ao encerrar a sessão. Isso resultou na perda de tentativas em uma execução intermediária que havia sido configurada para 70 tentativas e foi interrompida antes da conclusão. A solução adotada foi salvar o arquivo `.db` como dataset no painel de inputs do Kaggle antes do encerramento e copiá-lo para `/kaggle/working` ao iniciar novas sessões, permitindo que o Optuna retomasse o estudo do ponto em que parou.

2. Persistência e auditoria dos estudos. A adoção do salvamento contínuo via SQLite permitiu preservar três bancos de dados ao longo do projeto, todos referentes ao estudo `nas_ferplus`. O primeiro banco registra uma execução completa de 30 tentativas, concluída em 6 horas e 49 minutos. O segundo registra a execução ampliada, que atingiu 102 tentativas e consumiu mais de duas sessões completas de execução, o equivalente a aproximadamente 24 horas de GPU. O terceiro corresponde à continuação desse mesmo estudo até 107 tentativas, aproveitando o histórico acumulado pelo amostrador TPE. A preservação dos bancos possibilitou a auditoria posterior dos resultados apresentados na Subseção 4.1, com recuperação exata do melhor trial, da melhor acurácia média e dos hiperparâmetros de cada execução.

3. Uso de duas contas Kaggle. Para maximizar o tempo de GPU disponível, o projeto foi dividido entre duas contas, permitindo execuções em paralelo do NAS e do treino final em notebooks distintos.

4. Integração com W&B. A integração com o Weights and Biases foi adotada como solução definitiva para o problema de perda de modelos treinados, permitindo salvamento automático de artefatos na nuvem a cada melhoria de acurácia de validação.

3.6. Integração e Interface em Tempo Real

A integração final foi realizada em um script Python local (`reconhecimento_emocoes_webcam.py`) que combina os dois modelos e código-fonte completo disponível em: <https://github.com/osfarias/projetos/tree/main/reconhecimento-facial/fase-2>. O fluxo de execução por frame é o seguinte:

1. Captura do frame via OpenCV (`cv2.VideoCapture`)
2. Detecção de rostos pelo YOLOv8 com limiar de confiança de 0,25
3. Recorte de cada rosto detectado
4. Aplicação das transformações de pré-processamento (grayscale, resize 48x48, normalização)
5. Classificação da emoção pelo modelo CNN
6. Exibição do retângulo delimitador e da emoção com porcentagem de confiança na tela

Uma suavização temporal foi implementada para reduzir a oscilação do retângulo delimitador: em caso de falha de detecção em um frame, o último retângulo detectado é mantido por até 8 frames antes de ser descartado.

4. Resultados

4.1. Resultados do NAS

A Tabela 1 resume as três execuções do NAS cujos bancos de dados SQLite foram preservados, evidenciando a evolução do número de tentativas e o efeito das restrições de GPU da plataforma Kaggle sobre o desenho dos experimentos. Os valores foram extraídos diretamente dos bancos do Optuna, garantindo rastreabilidade dos resultados.

Tabela 1. Resumo das execuções do NAS com Optuna.

Execução	Tentativas	Duração	Melhor acc. média	Melhor trial
1	30	6h49min	81,41%	15
2	102	aprox. 24h	81,00%	99
3 (continuação da 2)	107	não registrada	81,15%	106

Na execução 1, o trial 15 propôs 4 camadas convolucionais com filtros 80, 112, 128 e 96, kernels 3, 3, 5 e 5, camada densa com 256 neurônios, dropout de 0,205 e taxa de aprendizado de 0,00154. Na execução 2, o trial 99 propôs 4 camadas convolucionais com filtros 16, 96, 128 e 128, kernels 3, 5, 5 e 5, camada densa com 192 neurônios, dropout de 0,258 e taxa de aprendizado de 0,00203. Na execução 3, que estendeu o mesmo estudo da execução 2, o trial 106 propôs configuração muito próxima, com filtros 16, 96, 112 e 128, mesmos kernels, mesma camada densa de 192 neurônios, dropout de 0,247 e taxa de aprendizado de 0,00137. A continuação registrada na execução 3 foi encerrada pelo esgotamento da cota semanal de GPU da plataforma, sem novas tentativas posteriores.

Três observações merecem destaque. Primeiro, as três execuções convergiram para o limite superior de profundidade do espaço de busca, com 4 camadas convolucionais, indicando que redes mais profundas dentro do espaço definido foram consistentemente favorecidas para o FERPlus. Segundo, as diferenças de acurácia média entre os

melhores trials das três execuções são inferiores a 0,5 ponto percentual, sugerindo um platô de desempenho para esse espaço de busca com validação cruzada de 3 folds e 3 épocas por fold. Terceiro, as arquiteturas levadas ao treino final derivaram das saídas do NAS, tanto de forma direta quanto por ajuste manual autorizado, conforme detalhado na Subseção 4.2.

4.2. Resultados do Treino Final

A auditoria dos checkpoints preservados, cruzada com os bancos de dados do Optuna, revelou que as versões comparadas na Tabela 2 não partiram de uma única arquitetura. A v2 utilizou diretamente a arquitetura do trial 15 da execução 1 do NAS (filtros 80, 112, 128 e 96, camada densa com 256 neurônios). As versões v1 e v4 utilizaram arquiteturas ajustadas manualmente pela equipe a partir das saídas do NAS, procedimento autorizado pelo professor orientador, que dispensou nova execução do Optuna. Em particular, a v4, de melhor desempenho, utilizou filtros 96, 96, 112 e 128, kernels 5, 3, 3 e 3, camada densa com 192 neurônios, dropout de 0,4 e weight decay de 0,0001. Os checkpoints das versões v3, v5 e v6 não foram preservados, tendo sido sobrescritos por execuções subsequentes em razão das restrições de sessão da plataforma.

Tabela 2. Comparativo das versões do treino final e suas arquiteturas.

Versão	Épocas	Regularização	Pesos/classe	Arquitetura	Acc. val.
v1	30	Básica	Não	ajuste manual	66,11%
v2	50	Básica	Não	trial 15 da execução 1	65,27%
v3	40	AdamW, Dropout2d, smoothing	Não	não preservada	66,63%
v4 (melhor)	40	AdamW, Dropout2d, smoothing	Sim	ajuste manual	66,78%
v5	40 (early stop 31)	Completa	Não	não preservada	66,63%
v6	80 (early stop 44)	Completa	Sim	não preservada	65,66%

O modelo v4 atingiu acurácia de validação de 66,78%, com diferença entre acurácia de treino (83%) e validação oscilando para menor em relação às demais versões. Na Figura 1, apesar da regularização completa, a v6 acionou o early stop após 15 épocas sem melhoras, com a diferença estabilizada em 16 pontos percentuais. Isso indica que as estratégias de regularização são eficazes na redução do overfitting, mas em alguma medida exigem microajustes, paciência e tempo de GPU. Vale destacar uma limitação metodológica deste trabalho: devido às restrições de quota de GPU impostas pelos ambientes de nuvem gratuitos utilizados (Kaggle), o histórico de treino época a época do modelo v4 não foi preservado, tendo sido sobrescrito por execuções subsequentes. Dessa forma, as curvas de loss e acurácia apresentadas nas Figuras 2 e 3 correspondem ao treino do modelo v6, único com histórico completo disponível, ainda que a v4 permaneça como a versão de melhor desempenho reportada na Tabela 2. Ainda assim, a acurácia de validação do modelo v4 permanece acessível através da extração direta do checkpoint salvo.

O resultado evidencia uma observação metodológica relevante: a acurácia média dos folds no NAS, obtida com apenas 3 épocas por tentativa, funcionou como indicador das tendências gerais do espaço de busca (profundidade de 4 camadas, filtros crescentes e camada densa enxuta), mas não como previsor da ordenação final entre arquiteturas. A arquitetura do trial 15, melhor pontuada nos folds com 81,41%, atingiu 65,27% no treino completo, enquanto o ajuste manual informado pelas tendências do NAS, combinado com regularização mais forte e pesos por classe, alcançou o melhor resultado, com 66,78% de acurácia de validação.

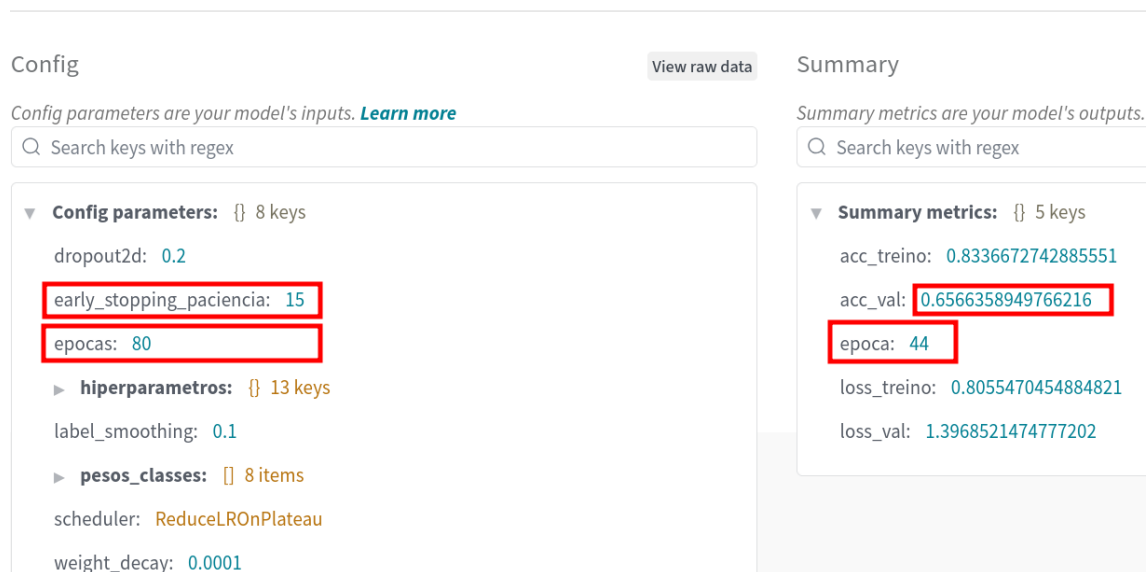


Figura 1. Modelo v6, dados salvos no W&B.

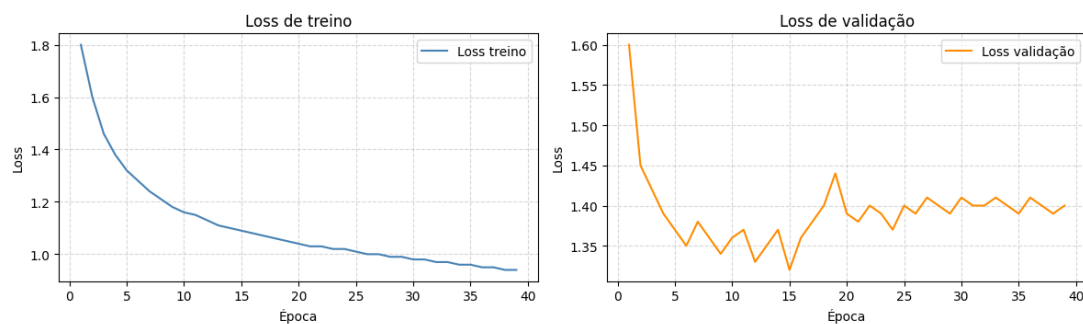


Figura 2. Curvas de loss de treino e validação do modelo v6.

4.3. Resultados da Detecção em Tempo Real

O sistema integrado foi testado em ambiente real com webcam, detectando múltiplos rostos simultaneamente. As emoções raiva, felicidade, neutralidade, nojo, desdém, surpresa e medo foram detectadas com sucesso durante os testes. A Figura 4 ilustra o sistema em funcionamento com vários rostos detectados simultaneamente a partir do posicionamento da webcam sobre uma imagem da internet.

A limitação mais relevante observada foi a dificuldade de detecção estável quando o rosto ocupa grande parte do frame. Esse comportamento foi atribuído ao fine-tuning inicial realizado exclusivamente com o WIDERFace, que contém predominantemente rostos de tamanho reduzido em imagens de multidões.

A Figura 5 ilustra uma detecção individual com a porcentagem de confiança exibida junto ao retângulo delimitador, comportamento descrito no fluxo de integração da Subseção 4.2.

Adicionalmente, a Figura 6 revela a capacidade do sistema de operar com várias pessoas em cena, mantendo a classificação individual de cada rosto. Nota-se que emoções distintas são atribuídas simultaneamente, sem interferência entre as detecções.

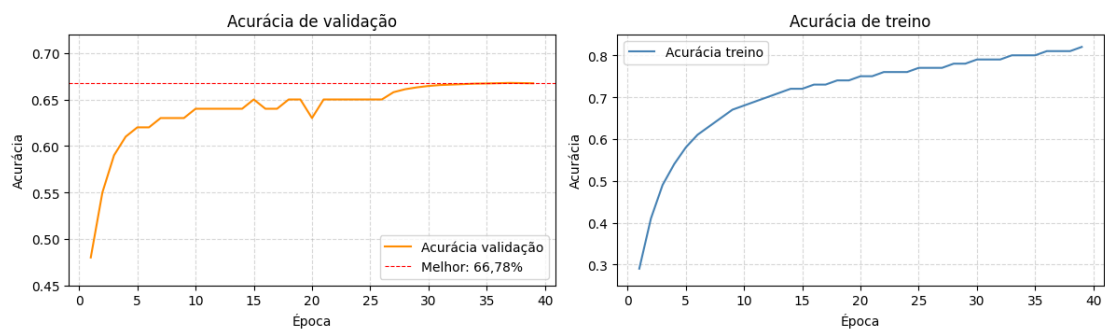


Figura 3. Curvas de acurácia de treino e validação do modelo v6.

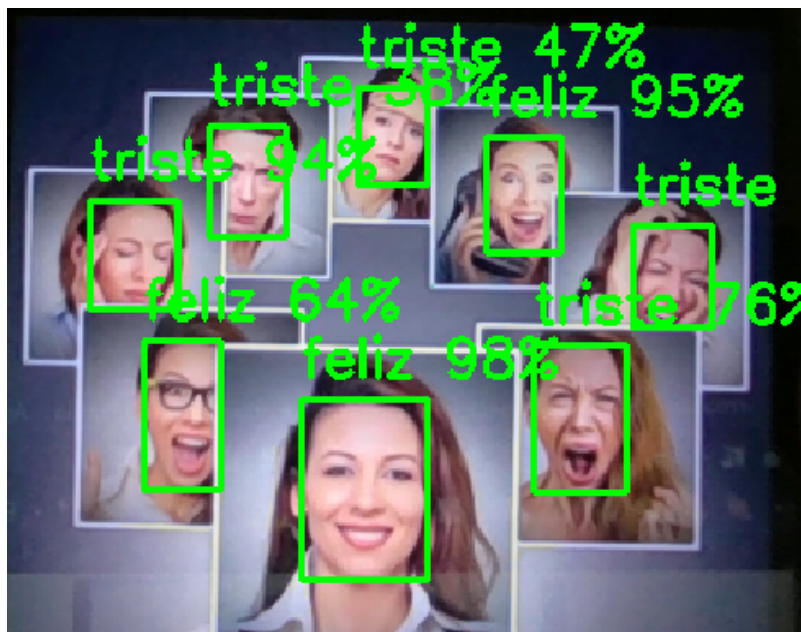


Figura 4. Sistema em funcionamento com detecção de múltiplos rostos e classificação de emoções em tempo real.

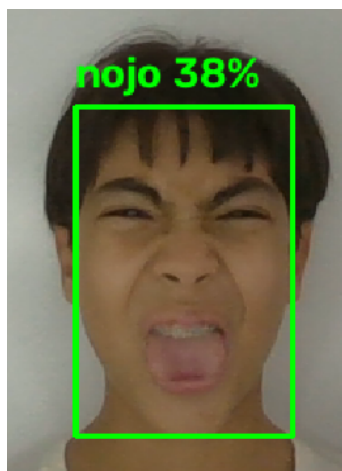


Figura 5. Exemplo de detecção da emoção nojo.

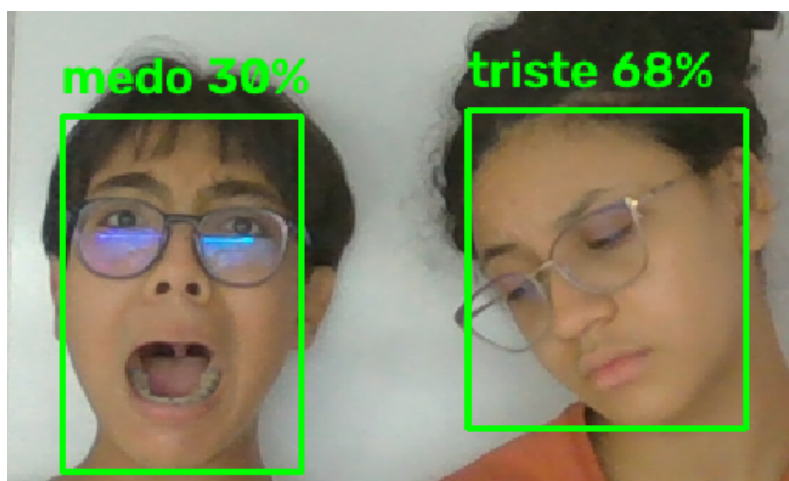


Figura 6. Exemplo de detecção de dois rostos simultaneamente.

Após a combinação com o Face-Detection-Dataset, a estabilidade do bounding box melhorou significativamente para rostos próximos à câmera.

5. Conclusão

Este trabalho apresentou o desenvolvimento completo de um sistema de reconhecimento de emoções faciais em tempo real, integrando Neural Architecture Search com Optuna, rede neural convolucional treinada do zero sobre o dataset FERPlus e YOLOv8 fine-tunado para detecção de rostos. A abordagem demonstrou ser viável no contexto de restrições de GPU em nuvem pública, com a plataforma Kaggle como ambiente principal de treinamento.

Os principais resultados obtidos foram: melhor arquitetura de CNN com acurácia média de 81,41% nos folds do NAS; melhor acurácia de validação de 66,78% no conjunto de validação do FERPlus, obtida com arquitetura ajustada manualmente a partir das tendências reveladas pelo NAS e treinada com regularização completa; e sistema integrado funcionando em tempo real com detecção de 7 das 8 emoções do dataset com múltiplos rostos simultâneos.

A auditoria dos bancos de dados do Optuna e dos checkpoints preservados evidenciou uma lição metodológica relevante: a acurácia média dos folds no NAS, obtida com poucas épocas por tentativa, funcionou como indicador das tendências gerais do espaço de busca, mas não como predictor da ordenação final entre arquiteturas, uma vez que o melhor desempenho no treino completo foi alcançado por uma variação manual informada por essas tendências, e não pelo melhor trial de nenhuma das execuções.

As estratégias de regularização adotadas progressivamente ao longo do projeto, em especial a combinação de Dropout2d, AdamW com weight decay, label smoothing, ReduceLROnPlateau e early stopping, foram determinantes para reduzir o overfitting observado nas versões iniciais, onde a diferença entre acurácia de treino e validação ultrapassava 30 pontos percentuais.

Os desafios práticos com restrições de GPU na plataforma Kaggle, como perda de sessões e limite de horas semanais, motivaram o desenvolvimento de estratégias de gerenciamento de estado como o salvamento contínuo do banco de dados do Optuna

via SQLite, a integração com o Weights and Biases para monitoramento e salvamento automático de modelos na nuvem, e o salvamento de checkpoints com arquitetura e hiperparâmetros embutidos, o que possibilitou a auditoria posterior e a rastreabilidade dos resultados reportados.

Como trabalhos futuros, identificam-se as seguintes direções: (1) implementação de mapas de calor para explicabilidade da detecção de rostos e da classificação de emoções, com ativação via tecla de atalho durante a demonstração em tempo real; (2) ampliação do número de tentativas do NAS para 200, com a estrutura de 150 tentativas com validação cruzada seguidas de 50 tentativas com o dataset completo sem divisão de folds; (3) exploração de arquiteturas com mecanismos de atenção sobre as regiões faciais mais relevantes para cada emoção, como olhos e boca; (4) avaliação do sistema em condições controladas de iluminação e distância para caracterização quantitativa do desempenho em ambiente real; e (5) treino completo das arquiteturas dos melhores trials das execuções preservadas, incluindo o trial 99 e o trial 106, para quantificar a relação entre a pontuação nos folds do NAS e o desempenho final.

Referências

- Akiba, T., Sano, S., Yanase, T., Ohta, T., and Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 2623–2631.
- Barsoum, E., Zhang, C., Canton Ferrer, C., and Zhang, Z. (2016). Training deep networks for facial expression recognition with crowd-sourced label distribution. In *Proceedings of the 18th ACM International Conference on Multimodal Interaction*, pages 279–283.
- Fang, B., Zhao, Y., Han, G., and He, J. (2023). Expression-guided deep joint learning for facial expression recognition. *Sensors*, 23(16):7148.
- Hua, W. et al. (2025). A novel facial expression recognition approach combining Canny edge detection and convolutional neural networks. *IET Software*.
- Terven, J. and Córdova-Esparza, D. (2023). A comprehensive review of YOLO architectures in computer vision: From YOLOv1 to YOLOv8 and YOLO-NAS. *Machine Learning and Knowledge Extraction*, 5(4):1680–1716.
- Wang, X. and Zhu, W. (2024). Advances in neural architecture search. *National Science Review*, 11(8):nwae282.
- White, C., Zela, A., Ru, B., Liu, Y., and Hutter, F. (2023). Neural architecture search: Insights from 1000 papers. *arXiv preprint*. arXiv:2301.08727.
- Wu, M.-T. and Tsai, C.-W. (2024). Training-free neural architecture search: A review. *ICT Express*, 10(1):213–231.
- Yang, S., Luo, P., Loy, C. C., and Tang, X. (2016). WIDER FACE: A face detection benchmark. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 5525–5533.
- Yisihak, H. M. and Li, L. (2024). Advanced face detection with YOLOv8: Implementation and integration into AI modules. *Scientific Research Publishing*.

- Zhao, F. et al. (2024). Introducing a novel dataset for facial emotion recognition and demonstrating significant enhancements in deep learning performance through pre-processing techniques. *Heliyon*, 10.
- Zhao, Z. et al. (2025). A perception CNN for facial expression recognition. *arXiv preprint*. arXiv:2512.06422.